

Submission Worksheet

Submission Data

Course: IT114-007-F2025

Assignment: IT114 Milestone 1

Student: Karen R. (kar65)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 11/3/2025 9:46:42 PM

Updated: 11/5/2025 3:55:38 AM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-1/grading/kar65>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-1/view/kar65>

Instructions

- Overview Link: <https://youtu.be/9dZPFwi76ak>

1. Refer to Milestone1 of any of these docs:
 2. [Rock Paper Scissors](#)
 3. [Basic Battleship](#)
 4. [Hangman / Word guess](#)
 5. [Trivia](#)
 6. [Go Fish](#)
 7. [Pictionary / Drawing](#)
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
 1. `git checkout Milestone1` (ensure proper starting branch)
 2. `git pull origin Milestone1` (ensure history is up to date)
5. Copy Part5 and rename the copy as Project (this new folder should be in the root of your repo)
6. Organize the files into their respective packages Client, Common, Server, Exceptions
 1. Hint: If it's open, you can refer to the Milestone 2 Prep lesson
7. Fill out the below worksheet
 1. Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 2. Since this Milestone was majorly done via lessons, the required comments should be placed in areas of analysis of the requirements in this worksheet. There shouldn't need to be any actual code changes beyond the restructure.
8. Once finished, click "Submit and Export"
9. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone1`
 4. On Github merge the pull request from Milestone1 to main
10. Upload the same PDF to Canvas
11. Sync Local

- ## Section #1: (1 pt.) Feature: Server Can Be Started Via Command Line And Listen To Connections

≡ Task #1 (1 pt.) - Evidence

Part 1:

- Show the terminal output of the server started and listening
- Show the relevant snippet of the code that waits for incoming connections

Screenshot of server listening on port 3000.

Code snippet showing the server listening on port 3000

⇒ Part 2:

Details:


```

63
64
65
66 //Karen Kaida //KARDS //11/04/2021
67
68 /**
69  * Callback passed to ServerThread to inform Server they're ready to receive
70  * data
71  */
72 @Override
73 private void onServerThreadInitialized(ServerThread serverThread) {
74     initLobby();
75     rooms.get("lobby").add(serverThread);
76     clientRoom.put(serverThread, "lobby");
77
78     roomLog(serverThread.getClientId() + "initialized; places in lobby");
79     // add to connected clients list (unique id and actual reference)
80     connectedClients.put(serverThread.getClientId(), serverThread);
81     relay(null, String.format("User[%d] connected", serverThread.getClientId()));
82 }
83
84 /**
85  * Takes a ServerThread and removes them from the server
86  * Adding the synchronized keyword ensures that only one thread can execute
87  * this method at a time
88  */

```

snippet of code of onServerThreadInitialized

```

1 import java.io.IOException;
2 import java.net.ServerSocket;
3 import java.net.Socket;
4 import java.util.concurrent.ConcurrentHashMap;
5 import java.util.concurrent.CopyOnWriteArrayList;
6
7 public class Server {
8     private int port = 3000;
9     // connected clients
10    // Use ConcurrentHashMap for thread safe client management
11    // The long will be a unique client identifier, and ServerThread is the instance
12    //KarenKaida //KARDS //Nov/04/2021
13    private final ConcurrentHashMap<Long, ServerThread> connectedClients = new ConcurrentHashMap<>();
14
15    private final ConcurrentHashMap<String, CopyOnWriteArrayList<ServerThread>> rooms = new ConcurrentHashMap<>();
16    private final ConcurrentHashMap<ServerThread, String> clientRoom = new ConcurrentHashMap<>();
17
18    private void initLobby() {
19        rooms.computeIfAbsent("lobby", k -> new CopyOnWriteArrayList<>());
20    }
21
22    private boolean isRunning = true;
23 }

```

snippet of code room fields

```

1 //Karen Kaida //KARDS //11/04/2021
2
3 // Callback passed to ServerThread to inform Server they're ready to receive
4 // data
5
6 @Override
7 private void onServerThreadInitialized(ServerThread serverThread) {
8     initLobby();
9     rooms.get("lobby").add(serverThread);
10    clientRoom.put(serverThread, "lobby");
11
12    roomLog(serverThread.getClientId() + "initialized; places in lobby");
13    // add to connected clients list (unique id and actual reference)
14    connectedClients.put(serverThread.getClientId(), serverThread);
15    relay(null, String.format("User[%d] connected", serverThread.getClientId()));
16 }
17
18 /**
19  * Takes a ServerThread and removes them from the server
20  * Adding the synchronized keyword ensures that only one thread can execute
21  * this method at a time
22  */

```

code snippet 1/2 that handles room management

```

1 //Karen Kaida //KARDS //11/04/2021
2
3 // Callback passed to ServerThread to inform Server they're ready to receive
4 // data
5
6 @Override
7 private void onServerThreadInitialized(ServerThread serverThread) {
8     initLobby();
9     rooms.get("lobby").add(serverThread);
10    clientRoom.put(serverThread, "lobby");
11
12    roomLog(serverThread.getClientId() + "initialized; places in lobby");
13    // add to connected clients list (unique id and actual reference)
14    connectedClients.put(serverThread.getClientId(), serverThread);
15    relay(null, String.format("User[%d] connected", serverThread.getClientId()));
16 }
17
18 /**
19  * Takes a ServerThread and removes them from the server
20  * Adding the synchronized keyword ensures that only one thread can execute
21  * this method at a time
22  */

```

code snippet of 2/2 that handle room management



Saved: 11/4/2025 7:29:42 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles room creation, joining/leaving, and removal

Your Response:

The Server manages chat rooms through thread safe maps that track the active rooms. When a client uses /createroom, the server checks whether a room exists and creates it if it doesn't. Using /join, user is moved from their current room to the selected room. The command /leaveroom command returns users to the default lobby.



Saved: 11/4/2025 7:29:42 PM

Section #4: (1 pt.) Feature: Client Can Be Started Via The Command Line

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the /name and /connect commands for each of 3 clients (best to use the split terminal feature)
- Output should show evidence of a successful connection
- Show the relevant snippets of code that handle the processes for /name, /connect, and the confirmation of being fully setup/connected

```
Karen@karenmashiki: ~/Kor05-IT
11-4-2025 (Milestone1)
Thread[26]: Thread starting
[room]initialized; places in lobby
Client connected
Thread[0]: ServerThread created
Waiting for next client
Thread[27]: Thread starting
[room]initialized; places in lobby
Client connected
Thread[0]: ServerThread created
Waiting for next client
Thread[28]: Thread starting
[room]initialized; places in lobby
Client connected

Kor05-IT114-007-2025 (Milestone1)
$ java -cp . Project.Client.Client
Client Created
Client starting
Waiting for input
/name Client1
[info] name set to Client1
/connect localhost:3000
Client connected
[info] connected to localhost:3000
Server: "User[26] connected"
Server: "User[27] connected"
Server: "User[28] connected"

Kor05-IT114-007-2025 (Milestone1)
$ java -cp . Project.Client.Client
Client Created
Client starting
Waiting for input
/name Client2
[info] name set to Client2
/connect localhost:3000
Client connected
[info] connected to localhost:3000
Server: "User[26] connected"
Server: "User[27] connected"
Server: "User[28] connected"
```

Snippet of terminal output showing three clients connected successfully.

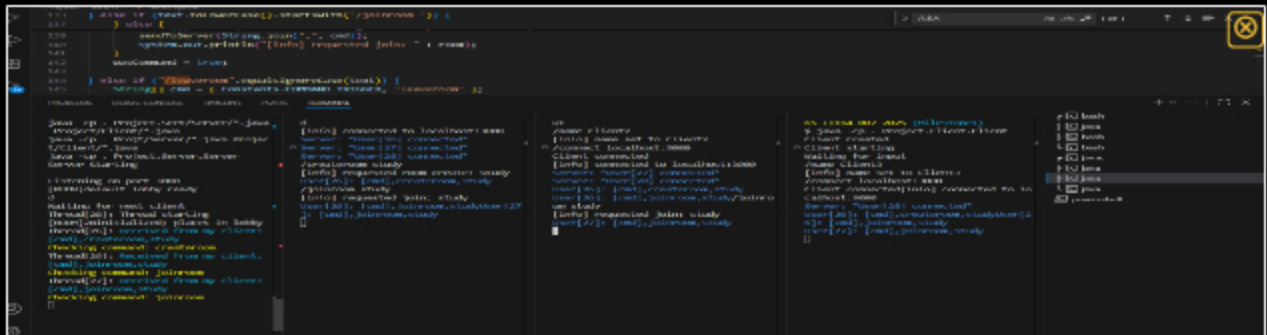
```
public class Client {
    private boolean processClientCommand(String text) throws IOException {
        if (text == null) return false;

        // Karen@karenmashiki: ~/Kor05-IT // 11/4/2025
        if (text.toLowerCase().startsWith("/name ")) {
            String[] parts = text.trim().replaceFirst("/name ", "").split(" ");
            String name = parts[0];
            if (name.length() > 0) {
                // Update client name
                clientName = name;
                // Send name to server
                String msg = "/name " + name;
                clientSocket.getOutputStream().write(msg.getBytes());
                clientSocket.getOutputStream().flush();
            }
        } else if (text.toLowerCase().startsWith("/connect ")) {
            String[] parts = text.trim().replaceFirst("/connect ", "").split(" ");
            String host = parts[0];
            int port = Integer.parseInt(parts[1]);
            // Connect to server
            clientSocket = new Socket(host, port);
            clientSocket.getOutputStream().write("Client connected".getBytes());
            clientSocket.getOutputStream().flush();
        }
    }
}
```

Progress: 100%

Details:

- Show the terminal output of the /createroom and /joinroom
- Output should show evidence of a successful creation/join in both scenarios
- Show the relevant snippets of code that handle the client-side processes for room creation and joining



Snippet terminal output of successful /create room and /joinroom



Snippet of code that handles the client-side processes for room and joining/



Saved: 11/4/2025 11:56:28 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the /createroom and /join room commands work and the related code flow for each

Your Response:

The commands /created and /joinroom work because when you type /createroom in the clients the processClientsCommand method receives the text and checks if it starts with the createroom or /joinroom commands. It then grabs the room name, it builds a small command message ToServer sends it. On the server side the client thread reads the message and detects the createroom command it creates the room or creates it if it does not exist.



Saved: 11/4/2025 11:56:28 PM

Section #6: (1 pt.) Feature: Client Can Send Messages

Progress: 100%

Task #1 (1 pt.) - Evidence

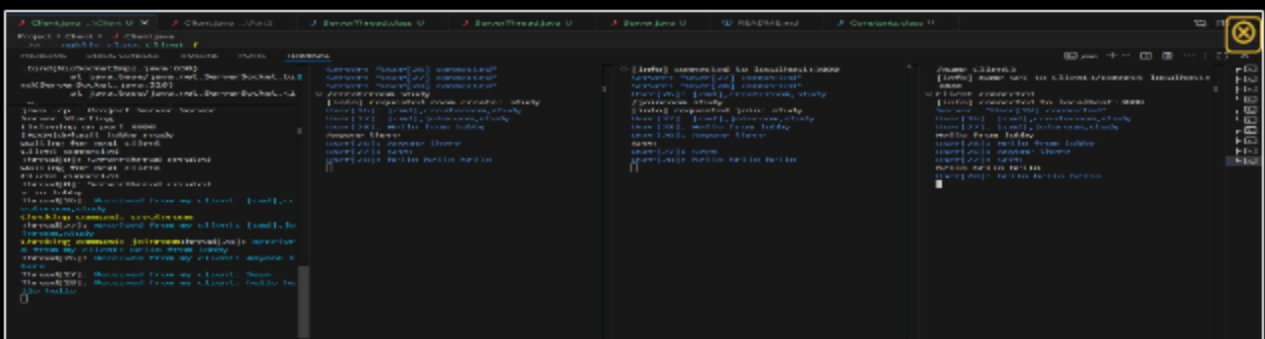
Progress: 100%

Part 1:

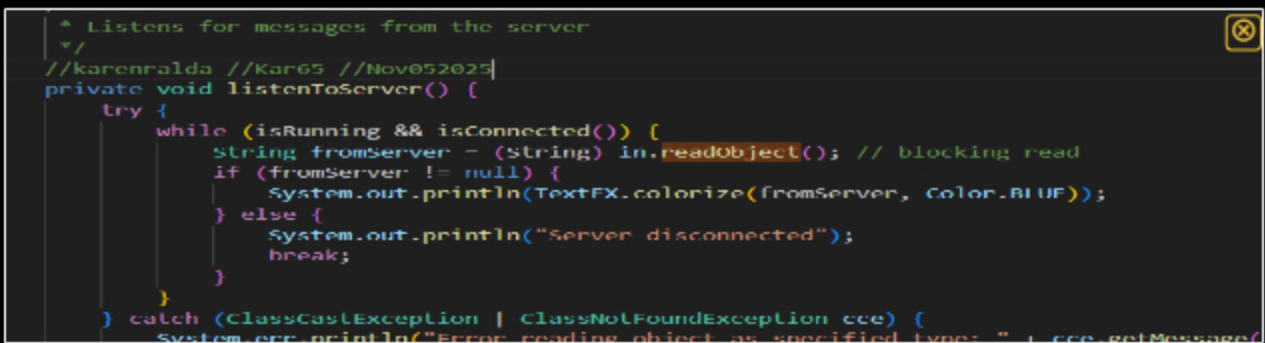
Progress: 100%

Details:

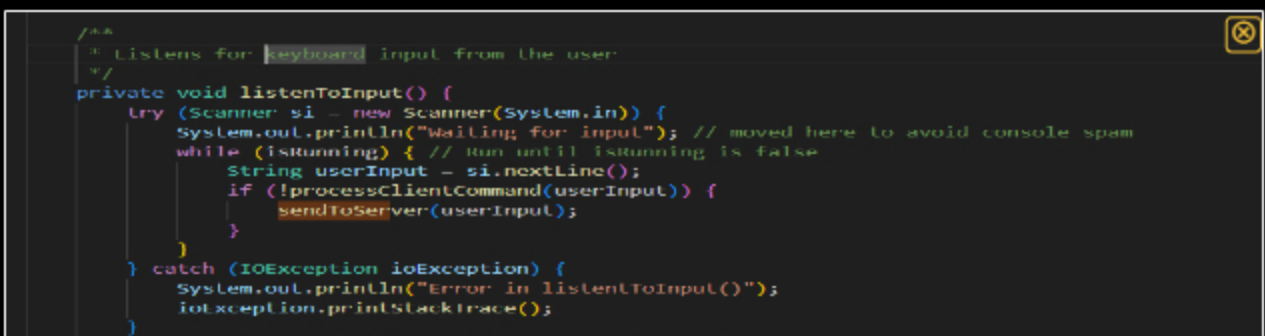
- Show the terminal output of a few messages from each of 3 clients
- Include examples of clients grouped into other rooms
- Show the relevant snippets of code that handle the message process from client to server-side and back



Screenshot of terminal messages from each 3 clients.



Client receives message from server snippet.



Snippet of client sending message to the server.

```
// client sends message to server
private void btnSendMessage_Click(object sender, EventArgs e)
{
    if (txtMessage.Text.Length > 0)
    {
        // send message to server
        client.Send(Encoding.UTF8.GetBytes(txtMessage.Text));
        txtMessage.Text = "";
    }
}

// server receives message and sends it back to the client
private void btnReceive_Click(object sender, EventArgs e)
{
    if (txtReceived.Length > 0)
    {
        // receive message from server
        byte[] data = new byte[1024];
        int received = client.Receive(data);
        string message = Encoding.UTF8.GetString(data, 0, received);
        txtReceived.AppendText(message + "\n");
    }
}
```

Snippet of server-side flow code



Saved: 11/5/2025 2:02:28 AM

Part 2:

Progress: 100%

Details:

- Briefly explain how the message code flow works

Your Response:

When a client types a message, the client side first checks for the commands like /createroom or /joinroom. If not a command the message is sent to the server using sendToServer.

On the server side the message arrives in serverThread.processpayload(). This method checks again if the message is a command. Relay function sends the message to every client currently in the same room.

The server keep track of the room each client is in.

On the client side, each client runs listenToServer() in a loop. The ReadObject waits for the message and print message onto the screen.



Saved: 11/5/2025 2:02:28 AM

Section #7: (1 pt.) Feature: Disconnection

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show examples of clients disconnecting (server should still be active)
- Show examples of server disconnecting (clients should be active but disconnected)
- Show examples of clients reconnecting when a server is brought back online
- Examples should include relevant messages of the actions occurring
- Show the relevant snippets of code that handle the client-side disconnection process
- Show the relevant snippets of code that handle the server-side termination process

Snippet of Client disconnecting while server stays active

Saved: 11/5/2025 2:59:22 AM

Part 2:

Progress: 100%

Details:

- Briefly explain how both client and server gracefully handle their disconnect/termination logic

Your Response:

When a client disconnects, it closes its socket and stops listening for messages. The server notices the connection is gone because the read loop fails and calls its disconnect() method for that client's thread. This removes the client from any room they were in and cleans up the server side sockets. The server continues to run so the client stays connected.

If the server disconnects the client's listening loop can no longer receive messages. The client program itself is still open, so the user can use /connect again once the server is online.

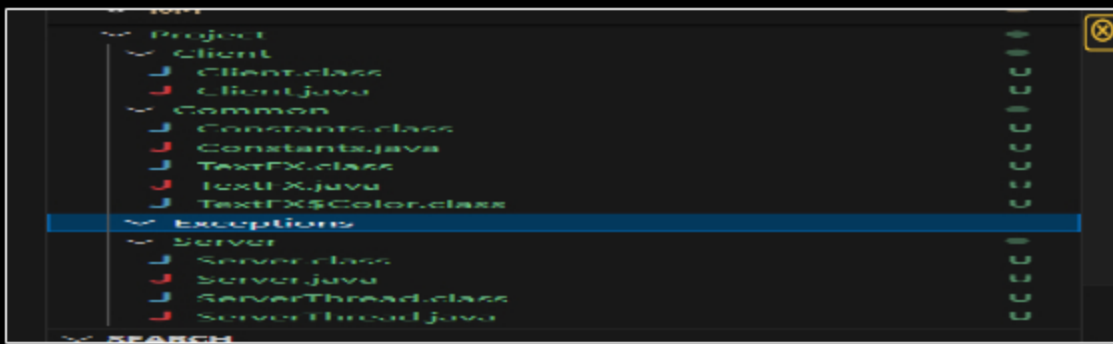
Saved: 11/5/2025 2:59:22 AM

Section #8: (1 pt.) Misc

Progress: 100%

Task #1 (0.25 pts.) - Show the proper workspace structure with the new Client, Common, Server, and Exceptions packages

Progress: 100%



Showing the proper workspace structure with the new client, common, server, and packages.

Saved: 11/5/2025 3:03:45 AM

☰ Task #2 (0.25 pts.) - Github Details

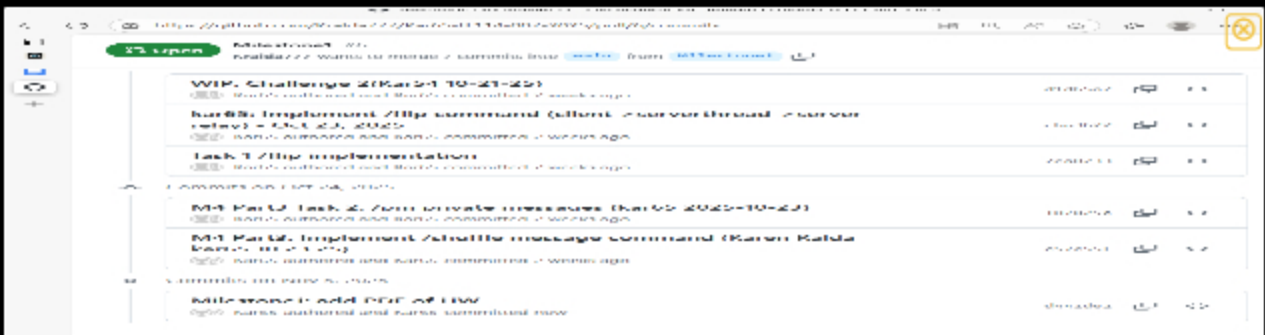
Progress: 100%

📁 Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history



Screenshot of Pull Request commit history

Saved: 11/5/2025 3:28:19 AM

🔗 Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/Kralda777/Kar65-IT114-007-2025/commits>



URL

<https://github.com/Kralda777/Kar65-IT114-007-2025/pull/1>

Saved: 11/5/2025 3:28:19 AM

Task #3 (0.25 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



Screenshot of overall time



Screenshot of individual time



Saved: 11/5/2025 3:33:18 AM

Task #4 (0.25 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

In milestone 1, I learned more about how client-server communication works using sockets in Java. I learned how the server waits to connect and how each client runs on its own thread which allows everyone to send and receive messages at the same time.

I learned how the messages travel from the client to the server and then the process of them getting sent to other clients. I learned how to handle disconnects so that the program does not crash. I got a better understanding of chat systems and networking programs and how they work.



Saved: 11/5/2025 3:44:28 AM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of this milestone was running the client and server from the command line once everything was already set up. /name and /connect commands were the easiest part for me.



Saved: 11/5/2025 3:46:55 AM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of this assignment was understanding and handling the disconnect. I was having trouble figuring out what the output was supposed to show and how the disconnection was supposed to look like. Section 7 was the toughest part for me making sure everything was shown for example the server still running when a client disconnects and then shows what happens when the

server stops and comes back online.. Getting my code to work is always the hardest for me takes me a while considering that the small mistakes can break connection. Disconnect and reconnect was the most challenging.



Saved: 11/5/2025 3:55:38 AM